



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ N 4

Название: «Реализация алгоритма шифрования с открытым ключом RSA»

Дисциплина: «Защита информации»

Студент ИУ7-72Б
(Группа)

(Подпись, дата)

Ю.А. Попов
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.С. Руденкова
(И. О. Фамилия)

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические сведения	4
1.1 Алгоритм RSA	4
1.2 КТО, подпись и гибридное шифрование	4
2 Ручной расчёт по варианту 15	6
3 Проектирование и реализация	7
3.1 Структура программы	7
3.2 Формат учебного RSA-файла	7
4 Тестирование	9
4.1 Методика	9
4.2 Измерение времени	10
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12
5 ПРИЛОЖЕНИЕ А. Листинг программы	13

ВВЕДЕНИЕ

Цель работы — реализовать алгоритм RSA и применить его для шифрования, расшифрования и контроля целостности файлов. В отличие от симметричного шифрования, RSA использует два ключа: открытый ключ можно передавать всем пользователям, а закрытый ключ остаётся у владельца. Стойкость RSA связывают со сложностью разложения большого числа на простые множители [1].

Для достижения цели решены следующие задачи:

- 1) реализованы тест Миллера–Рабина, расширенный алгоритм Евклида и быстрое возведение в степень по модулю;
- 2) реализованы генерация, сохранение и загрузка пары RSA-ключей;
- 3) реализованы шифрование целого числа, текста UTF-8 и произвольного файла;
- 4) применена китайская теорема об остатках при расшифровании;
- 5) реализованы цифровая подпись SHA-256/RSA, гибридная схема и проверка повреждения файлов;
- 6) проведены проверки граничных случаев и замеры времени.

1 Теоретические сведения

1.1 Алгоритм RSA

Для генерации ключей выбираются различные простые числа p и q . Затем вычисляются модуль n и функция Эйлера $\varphi(n)$:

$$n = p \cdot q, \quad \varphi(n) = (p - 1)(q - 1). \quad (1.1)$$

Открытая экспонента e должна быть взаимно простой с $\varphi(n)$. Закрытая экспонента d является обратной к e по модулю $\varphi(n)$, то есть $ed \equiv 1 \pmod{\varphi(n)}$. Открытый ключ имеет вид (n, e) , закрытый — (n, d) . Для сообщения M , удовлетворяющего $0 \leq M < n$, выполняются преобразования

$$C = M^e \pmod{n}, \quad M = C^d \pmod{n}. \quad (1.2)$$

Корректность расшифрования следует из теоремы Эйлера и выбора d . Показатель $e = 65537$ выбран в программной реализации, поскольку он достаточно мал для быстрого шифрования и обычно взаимно прост с $\varphi(n)$.

Для вычисления степеней реализован метод квадратов и умножений. Он обрабатывает биты показателя и требует $O(\log e)$ модульных умножений, а не $O(e)$. Кандидаты на простые числа проверяются 40 раундами Миллера–Рабина [2]. Случайные значения формируются модулем `secrets`, а не генератором общего назначения.

1.2 КТО, подпись и гибридное шифрование

При наличии p и q расшифрование можно выполнить отдельно по модулям p и q , после чего объединить результаты китайской теоремой об остатках. В реализации предварительно вычисляется $q^{-1} \pmod{p}$; это уменьшает размер операндов модульного возведения в степень примерно вдвое.

Учебный RSA не имеет дополнения и детерминирован: одинаковые блоки дают одинаковый шифртекст. Кроме того, для него выполняется мультиплика-

тивное свойство $E(M_1)E(M_2) \equiv E(M_1 M_2) \bmod n$. Поэтому он непригоден для практической защиты данных без дополнительной схемы. В гибридном режиме файл шифруется AES-256-CTR, а случайный 256-битный сеансовый ключ защищается RSAES-OAEP. AES стандартизован в FIPS 197 [3], а OAEP описан в PKCS N 1 [4].

Подпись строится как RSA-преобразование хэша SHA-256 исходного файла. При проверке подпись преобразуется открытым ключом и сравнивается с заново вычисленным хэшем. SHA-256 определён стандартом Secure Hash Standard [5].

2 Ручной расчёт по варианту 15

Для варианта 15 заданы $p = 83$, $q = 19$, $e = 13$ и $M = 222$. Длина ключа для файлового эксперимента составляет 3072 бита, а тип файла — документ Word. Из определения параметров RSA получаются значения

$$n = 83 \cdot 19 = 1577, \quad \varphi(n) = 82 \cdot 18 = 1476. \quad (2.1)$$

Проверка взаимной простоты и расширенный алгоритм Евклида имеют вид:

$$1476 = 113 \cdot 13 + 7; \quad 13 = 1 \cdot 7 + 6; \quad 7 = 1 \cdot 6 + 1. \quad (2.2)$$

Отсюда $1 = 2 \cdot 1476 - 227 \cdot 13$, поэтому $d = -227 \bmod 1476 = 1249$. Следовательно, $13 \cdot 1249 \equiv 1 \bmod 1476$.

Быстрое возведение сообщения в степень:

$$222 \bmod 1577 = 397; \quad 222^4 \bmod 1577 = 1486; \quad 222^8 \bmod 1577 = 396; \quad 222^{13} \bmod 1577 = 396 \cdot 1486 \cdot 222 \bmod 1577 = 129.$$

Получен шифртекст $C = 129$. Программа проверяет результат командой, приведённой в листинге 2.1. Расшифрование возвращает 222, что подтверждает корректность ручного расчёта.

```
1 $ python3 src/main.py variant15
2 {
3   "n": 1577,
4   "phi": 1476,
5   "d": 1249,
6   "ciphertext": 129,
7   "decrypted": 222
8 }
```

Листинг 2.1 — Проверка расчёта варианта 15 программой

3 Проектирование и реализация

3.1 Структура программы

Исходные файлы находятся в каталоге `src`. Назначение их компонентов приведено в таблице 3.1. В реализации не используются готовые криптографические библиотеки.

Таблица 3.1 — Компоненты программы

Файл	Основные функции	Назначение
<code>rsa_crypto.py</code>	<code>extended_gcd</code> , <code>mod_pow</code> , <code>is_probable_prime</code>	Математические операции RSA, обратный элемент, Миллер–Рабин.
<code>rsa_crypto.py</code>	<code>generate_key_pair</code> , <code>encrypt_integer</code> , <code>decrypt_integer</code>	Генерация ключей и операции над целыми числами; КТО.
<code>rsa_crypto.py</code>	<code>encrypt_blocks</code> , <code>decrypt_blocks</code>	Обязательное поблочное шифрование произвольного файла.
<code>rsa_crypto.py</code>	<code>sign_file</code> , <code>verify_file</code>	Подпись SHA-256/RSA и проверка файла.
<code>rsa_crypto.py</code>	<code>encrypt_file</code> , <code>decrypt_file</code>	Гибридное AES-256-CTR и RSA-ОАЕР шифрование.
<code>main.py</code>	Команды CLI	Ввод путей, обработка ошибок и вывод результатов.

Открытый ключ сохраняется в JSON-контейнере `lab04-public-v1` с полями n и e . Закрытый ключ помещается в `lab04-private-v1`: он защищён ключом, полученным PBKDF2-HMAC-SHA256 из мастер-пароля, и HMAC-тегом. Использование `hashlib.pbkdf2_hmac` соответствует документации Python [6].

3.2 Формат учебного RSA-файла

При длине модуля k байт открытый текст разбивается на блоки длиной k — 1. Число такого блока всегда меньше n , а шифртекст каждого блока занимает ровно k байт. Формат файла RSA-L4 показан в таблице 3.2. Длина исходных данных позволяет восстановить последний неполный блок; контрольная сумма SHA-256 выявляет повреждение шифртекста до записи результата.

Таблица 3.2 — Формат контейнера учебного поблочного RSA

Поле	Размер	Назначение
magic	8 байт	Сигнатура и версия RSA-L4.
block_size	2 байта	Размер открытого блока $k - 1$.
original_length	8 байт	Исходная длина в байтах.
key_fingerprint	32 байта	SHA-256 от открытого ключа; обнаруживает чужой ключ.
digest	32 байта	SHA-256 исходного файла; обнаруживает искажение.
cipher_blocks	k байт каждый	Последовательность блоков RSA.

Основные команды для документа варианта 15 приведены в листинге 3.1. Значение `--bits 3072` можно не указывать, поскольку это значение выбрано по умолчанию для варианта 15.

```

1 python3 src/main.py genkeys --bits 3072 --pub public.key --priv private.key
2 python3 src/main.py encrypt --key public.key --in source.docx --out source.rsa
3 python3 src/main.py decrypt --key private.key --in source.rsa --out restored.docx
4 python3 src/main.py sign --key private.key --in source.docx --out source.sig
5 python3 src/main.py verify --key public.key --in source.docx --out source.sig
6 python3 src/main.py hybrid-encrypt --key public.key --in source.docx --out source.hybrid

```

Листинг 3.1 — Команды консольного интерфейса

4 Тестирование

4.1 Методика

Автоматические тесты запускаются командой `python3 -m unittest discover -s tests -v`. Проверены AES-контрольный вектор, защита закрытого ключа паролем, гибридное шифрование файлов длиной 0, 1 и 5120 байт, а также подпись и изменение контейнера. Дополнительно вручную проверен формат RSA-L4 на файле с сигнатурой Word PK, на повреждённом шифртексте, строке с кириллицей и двух вариантах расшифрования: обычном и с КТО. Результаты приведены в таблице 4.1.

Таблица 4.1 — Результаты функциональных проверок

N	Сценарий	Результат
1	Контрольный вектор AES-256.	Совпадает с эталонным блоком FIPS 197.
2	Верный и неверный мастер-пароль.	Ключ восстанавливается либо выдаётся понятная ошибка.
3	Пустой, однобайтный и большой файл.	Данные расшифровываются побайтно без потерь.
4	Файл Word-подобного формата и последний неполный блок.	Исходный и восстановленный файлы совпадают.
5	Повреждение блока либо применение чужого ключа.	Расшифрование прерывается до записи выходного файла.
6	Изменение подписанного файла.	Подпись признаётся некорректной.
7	Строка с кириллицей и символом Unicode.	Текст UTF-8 восстанавливается точно.

Команда тестирования завершилась сообщением `Ran 4 tests ... OK`. Отдельная проверка файла варианта 15 завершилась сообщением `variant-15 DOCX-like file and integrity check: OK`. Таким образом, поддерживаются пустые файлы, частичный последний блок и обработка ошибочных данных.

4.2 Измерение времени

Замеры выполнены на случайном файле размером 100 КиБ. Время включает только указанную операцию; для 3072-битного ключа генерация и сохранение заняли 6,2 с. Результаты приведены в таблице 4.2.

Таблица 4.2 — Замеры поблочного RSA на файле 100 КиБ

Длина ключа, бит	Генерация, с	Шифрование, с	Обычное, с	КТО, с
1024	0,18	0,05	4,41	1,80
2048	0,85	0,08	15,05	4,64
3072	6,20	0,11	28,97	8,42

С увеличением ключа растут стоимость генерации простых чисел и время расшифрования. Шифрование остаётся существенно быстрее, поскольку показатель $e = 65537$ короток. Применение КТО ускорило расшифрование примерно в 2,4–3,4 раза: операции выполняются с модулями p и q примерно вдвое меньшей длины.

Команда `weaknesses --key public.key` демонстрирует детерминированность и мультипликативное свойство учебной схемы. Это объясняет, почему для реальных данных добавлен отдельный гибридный режим с ОАЕР, случайным сеансовым ключом и HMAC.

ЗАКЛЮЧЕНИЕ

В ходе работы реализован RSA на Python без криптографических библиотек. Написаны собственные алгоритмы быстрого модульного возведения в степень, расширенного Евклида, вычисления обратного элемента и проверки простоты Миллера–Рабина. Для варианта 15 вручную и программно получены $n = 1577$, $\varphi(n) = 1476$, $d = 1249$ и $C = 129$.

Программа обрабатывает произвольные двоичные файлы, включая документы Word, проверяет ключ и целостность контейнера, поддерживает подпись SHA-256/RSA и гибридное шифрование. Измерения на файле 100 КиБ подтвердили преимущество расшифрования с КТО. Проверки подтвердили обратимость преобразования и корректную реакцию на неверный пароль, чужой ключ и повреждённый шифртекст.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Rivest R. L., Shamir A., Adleman L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems // Communications of the ACM. — 1978. — Т. 21, № 2. — С. 120–126.
2. Miller G. L. Riemann's Hypothesis and Tests for Primality // Journal of Computer and System Sciences. — 1976. — Т. 13, № 3. — С. 300–317.
3. Technology. FIPS 197: Advanced Encryption Standard (AES) [Электронный ресурс]. — 2023. — Режим доступа: <https://csrc.nist.gov/pubs/fips/197/final>.
4. Moriarty K.; Kaliski B.; Jonsson J.; Rusch A. PKCS N 1: RSA Cryptography Specifications Version 2.2 [Электронный ресурс]. — 2016. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc8017.html>.
5. Technology. FIPS 180-4: Secure Hash Standard [Электронный ресурс]. — 2015. — Режим доступа: <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>.
6. Python Software Foundation. hashlib: Secure hashes and message digests [Электронный ресурс]. — 2026. — Режим доступа: https://docs.python.org/3/library/hashlib.html#hashlib.pbkdf2_hmac.

5 ПРИЛОЖЕНИЕ А. Листинг программы

```
"""Лабораторная работа №4: RSA без сторонних криптографических библиотек.

`encrypt_blocks` -- обязательный учебный блочный RSA (детерминированный).
`encrypt_file` -- гибридная демонстрация: AES-256-CTR + RSA-OAEP + HMAC.
"""

from __future__ import annotations

import base64
import hashlib
import hmac
import json
import math
import secrets
import struct
import tempfile
import time

from dataclasses import dataclass
from pathlib import Path
from typing import Any

SHA256_SIZE = 32
PBKDF2_ITERATIONS = 310_000
OAEP_LABEL = b"lab04 RSA session key"
RAW_MAGIC = b"RSA-L4\x01\x00"

# magic, plaintext block size, original length, recipient key fingerprint,
# SHA-256 of the original bytes. The last field detects damaged raw-RSA data.
RAW_HEADER = struct.Struct(">8sHQ32s32s")

VARIANT_15 = {
    "p": 83,
    "q": 19,
    "e": 13,
    "message": 222,
    "bits": 3072,
    "file_type": ".docx",
}

def _b64(v: bytes) -> str:
    return base64.b64encode(v).decode("ascii")

def _unb64(v: str) -> bytes:
    return base64.b64decode(v.encode("ascii"), validate=True)

def _os2ip(v: bytes) -> int:
    return int.from_bytes(v, "big")
```

```

def _i2osp(v: int, n: int) -> bytes:
    if not 0 <= v < 256**n:
        raise ValueError("integer does not fit requested length")
    return v.to_bytes(n, "big")

def _xor(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
    """Расширенный алгоритм Евклида: возвращает gcd, x, y: ax+by=gcd."""
    r0, r1, x0, x1, y0, y1 = a, b, 1, 0, 0, 1
    while r1:
        q = r0 // r1
        r0, r1, x0, x1, y0, y1 = r1, r0 - q * r1, x1, x0 - q * x1, y1, y0 - q * y1
    return r0, x0, y0

def mod_inverse(a: int, n: int) -> int:
    g, x, _ = extended_gcd(a, n)
    if g != 1:
        raise ValueError("modular inverse does not exist")
    return x % n

def mod_pow(a: int, exponent: int, n: int) -> int:
    """Собственное быстрое возведение в степень (square-and-multiply)."""
    if exponent < 0 or n <= 0:
        raise ValueError("invalid exponent or modulus")
    result, a = 1 % n, a % n
    while exponent:
        if exponent & 1:
            result = result * a % n
        a = a * a % n
        exponent >>= 1
    return result

def is_probable_prime(n: int, rounds: int = 40) -> bool:
    if n < 2:
        return False
    small = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)
    if n in small:
        return True
    if any(n % p == 0 for p in small):
        return False
    d, s = n - 1, 0
    while d % 2 == 0:
        d, s = d // 2, s + 1
    for _ in range(rounds):

```

```

a = secrets.randbelow(n - 3) + 2
x = mod_pow(a, d, n)
if x in (1, n - 1):
    continue
for _ in range(s - 1):
    x = x * x % n
    if x == n - 1:
        break
    else:
        return False
return True

def generate_prime(bits: int, rounds: int = 40) -> int:
    if bits < 2:
        raise ValueError("prime must have at least 2 bits")
    while True:
        n = secrets.randbits(bits) | 1 | (1 << (bits - 1))
        if is_probable_prime(n, rounds):
            return n

@dataclass(frozen=True)
class PublicKey:
    n: int
    e: int = 65537

    @property
    def size_bytes(self) -> int:
        return (self.n.bit_length() + 7) // 8

    def to_dict(self) -> dict[str, str]:
        return {"n": str(self.n), "e": str(self.e)}

    @classmethod
    def from_dict(cls, v: dict[str, str]) -> "PublicKey":
        key = cls(int(v["n"]), int(v["e"]))
        if key.n <= 2 or not 1 < key.e < key.n:
            raise ValueError("invalid public RSA key")
        return key

@dataclass(frozen=True)
class PrivateKey:
    n: int
    d: int
    e: int = 65537
    p: int | None = None
    q: int | None = None

    @property
    def public_key(self) -> PublicKey:

```

```

        return PublicKey(self.n, self.e)

    @property
    def size_bytes(self) -> int:
        return (self.n.bit_length() + 7) // 8

    def to_dict(self) -> dict[str, str]:
        return {
            "n": str(self.n),
            "d": str(self.d),
            "e": str(self.e),
            "p": str(self.p or ""),
            "q": str(self.q or ""),
        }

    @classmethod
    def from_dict(cls, v: dict[str, str]) -> "PrivateKey":
        key = cls(
            int(v["n"]),
            int(v["d"]),
            int(v["e"]),
            int(v["p"]) if v.get("p") else None,
            int(v["q"]) if v.get("q") else None,
        )
        if (
            key.n <= 2
            or not 0 < key.d < key.n
            or (key.p is None) != (key.q is None)
            or (key.p and key.p * key.q != key.n)
        ):
            raise ValueError("invalid private RSA key")
        return key

def generate_key_pair(bits: int = 2048) -> tuple[PublicKey, PrivateKey]:
    if bits < 1024 or bits % 2:
        raise ValueError("RSA size must be even and at least 1024 bits")
    while True:
        p, q = generate_prime(bits // 2), generate_prime(bits // 2)
        phi = (p - 1) * (q - 1)
        if p != q and math.gcd(65537, phi) == 1:
            n = p * q
    return PublicKey(n), PrivateKey(n, mod_inverse(65537, phi), 65537, p, q)

def encrypt_integer(message: int, key: PublicKey) -> int:
    if not 0 <= message < key.n:
        raise ValueError("message must satisfy 0 <= M < n")
    return mod_pow(message, key.e, key.n)

def decrypt_integer(ciphertext: int, key: PrivateKey, *, use_crt: bool = True) -> int:

```

```

        if not 0 <= ciphertext < key.n:
            raise ValueError("ciphertext must satisfy 0 <= C < n")
        if use_crt and key.p and key.q:
            return _decrypt_crt(ciphertext, key, mod_inverse(key.q, key.p))
        return mod_pow(ciphertext, key.d, key.n)

def _decrypt_crt(ciphertext: int, key: PrivateKey, q_inverse: int) -> int:
    """RSA private operation with precomputed  $q^{-1} \bmod p$ ."""
    assert key.p is not None and key.q is not None
    mp = mod_pow(ciphertext, key.d % (key.p - 1), key.p)
    mq = mod_pow(ciphertext, key.d % (key.q - 1), key.q)
    return mq + key.q * ((mp - mq) * q_inverse % key.p)

def solve_rsa_example(p: int, q: int, e: int, message: int) -> dict[str, int]:
    """Calculate a small RSA example, including the inverse via EEA.

    It is intended to check the manual calculation from task 1, not to create
    production keys (small p and q are insecure).
    """
    if not is_probable_prime(p) or not is_probable_prime(q) or p == q:
        raise ValueError("p and q must be distinct prime numbers")
    n = p * q
    phi = (p - 1) * (q - 1)
    if math.gcd(e, phi) != 1:
        raise ValueError("e is not coprime with phi(n)")
    d = mod_inverse(e, phi)
    public = PublicKey(n, e)
    private = PrivateKey(n, d, e, p, q)
    ciphertext = encrypt_integer(message, public)
    return {
        "n": n,
        "phi": phi,
        "d": d,
        "ciphertext": ciphertext,
        "decrypted": decrypt_integer(ciphertext, private),
    }

def solve_variant_15() -> dict[str, int]:
    """Return the program check for the assigned variant 15."""
    return solve_rsa_example(83, 19, 13, 222)

def rsa_decrypt_timing(
    ciphertext: int, key: PrivateKey, repetitions: int = 3
) -> tuple[float, float]:
    if repetitions < 1:
        raise ValueError("repetitions must be positive")
    t = time.perf_counter()
    for _ in range(repetitions):

```

```

    decrypt_integer(ciphertext, key, use_crt=False)
    normal = (time.perf_counter() - t) / repetitions
    t = time.perf_counter()
    for _ in range(repetitions):
        decrypt_integer(ciphertext, key, use_crt=True)
    return normal, (time.perf_counter() - t) / repetitions

# Minimal AES-256 encryption primitive (FIPS-197), used by hybrid mode only.
def _gmul(a: int, b: int) -> int:
    r = 0
    for _ in range(8):
        if b & 1:
            r ^= a
        a = ((a << 1) ^ (0x11B if a & 128 else 0)) & 255
        b >>= 1
    return r

def _gf_pow(a: int, p: int) -> int:
    r = 1
    while p:
        if p & 1:
            r = _gmul(r, a)
        a = _gmul(a, a)
        p >>= 1
    return r

def _rot8(a: int, s: int) -> int:
    return ((a << s) | (a >> (8 - s))) & 255

def _sbox(a: int) -> int:
    b = 0 if not a else _gf_pow(a, 254)
    return b ^ _rot8(b, 1) ^ _rot8(b, 2) ^ _rot8(b, 3) ^ _rot8(b, 4) ^ 0x63

_SBOX = tuple(_sbox(i) for i in range(256))

def _expand_aes256_key(key: bytes) -> list[bytes]:
    if len(key) != 32:
        raise ValueError("AES-256 key must contain 32 bytes")
    w = [list(key[i : i + 4]) for i in range(0, 32, 4)]
    rcon = 1
    for i in range(8, 60):
        t = w[i - 1].copy()
        if i % 8 == 0:
            t = [_SBOX[x] for x in t[1:] + t[:1]]
            t[0] ^= rcon
            rcon = _gmul(rcon, 2)

```

```

        elif i % 8 == 4:
            t = [_SBOX[x] for x in t]
            w.append([a ^ b for a, b in zip(w[i - 8], t)])
    return [bytes(sum(w[i : i + 4], [])) for i in range(0, 60, 4)]

def _aes_block(block: bytes, keys: list[bytes]) -> bytes:
    if len(block) != 16:
        raise ValueError("AES operates on 16-byte blocks")
    s = [x ^ keys[0][i] for i, x in enumerate(block)]
    for rnd in range(1, 15):
        s = [_SBOX[x] for x in s]
        old = s.copy()
        for r in range(4):
            for c in range(4):
                s[4 * c + r] = old[4 * ((c + r) % 4) + r]
            if rnd < 14:
                for c in range(4):
                    i = 4 * c
                    a, b, d, e = s[i : i + 4]
                    s[i : i + 4] = [
                        _gmul(a, 2) ^ _gmul(b, 3) ^ d ^ e,
                        a ^ _gmul(b, 2) ^ _gmul(d, 3) ^ e,
                        a ^ b ^ _gmul(d, 2) ^ _gmul(e, 3),
                        _gmul(a, 3) ^ b ^ d ^ _gmul(e, 2),
                    ]
        s = [x ^ keys[rnd][i] for i, x in enumerate(s)]
    return bytes(s)

def aes256_ctr(data: bytes, key: bytes, initial_counter: bytes) -> bytes:
    if len(initial_counter) != 16:
        raise ValueError("CTR counter must contain 16 bytes")
    keys = _expand_aes256_key(key)
    ctr = int.from_bytes(initial_counter, "big")
    out = bytearray()
    for i in range(0, len(data), 16):
        out.extend(_xor(data[i : i + 16], _aes_block(ctr.to_bytes(16, "big"), keys)))
        ctr = (ctr + 1) % (1 << 128)
    return bytes(out)

def _mgf1(seed: bytes, length: int) -> bytes:
    return b"".join(
        hashlib.sha256(seed + _i2osp(i, 4)).digest()
        for i in range(math.ceil(length / SHA256_SIZE))
   )[:length]

def rsa_oaep_encrypt(
    message: bytes, key: PublicKey, label: bytes = OAEP_LABEL
) -> bytes:

```

```

        k = key.size_bytes
        if len(message) > k - 2 * SHA256_SIZE - 2:
            raise ValueError("message is too long for RSA-OAEP key")
        db = (
            hashlib.sha256(label).digest()
            + b"\0" * (k - len(message) - 2 * SHA256_SIZE - 2)
            + b"\1"
            + message
        )
        seed = secrets.token_bytes(SHA256_SIZE)
        masked_db = _xor(db, _mgf1(seed, k - SHA256_SIZE - 1))
        encoded = b"\0" + _xor(seed, _mgf1(masked_db, SHA256_SIZE)) + masked_db
        return _i2osp(encrypt_integer(_os2ip(encoded), key), k)

    def rsa_oaep_decrypt(
        cipher: bytes, key: PrivateKey, label: bytes = OAEP_LABEL
    ) -> bytes:
        k = key.size_bytes
        if len(cipher) != k or _os2ip(cipher) >= key.n:
            raise ValueError("invalid RSA-OAEP ciphertext")
        encoded = _i2osp(decrypt_integer(_os2ip(cipher), key), k)
        ms, mdb = encoded[1:33], encoded[33:]
        seed = _xor(ms, _mgf1(mdb, 32))
        db = _xor(mdb, _mgf1(seed, k - 33))
        sep = db.find(b"\1", 32)
        if (
            encoded[0]
            or not hmac.compare_digest(db[:32], hashlib.sha256(label).digest())
            or sep < 32
            or any(db[32:sep])
        ):
            raise ValueError("invalid RSA-OAEP ciphertext")
        return db[sep + 1 :]

    def _auth_encrypt(
        data: bytes, key: bytes, context: bytes
    ) -> tuple[bytes, bytes, bytes]:
        nonce = secrets.token_bytes(16)
        cipher = aes256_ctr(data, key, nonce)
        tag = hmac.new(
            hashlib.sha256(b"mac" + key).digest(), context + nonce + cipher, hashlib.sha256
        ).digest()
        return nonce, cipher, tag

    def _auth_decrypt(
        nonce: bytes, cipher: bytes, tag: bytes, key: bytes, context: bytes
    ) -> bytes:
        expected = hmac.new(
            hashlib.sha256(b"mac" + key).digest(), context + nonce + cipher, hashlib.sha256

```

```

        ).digest()
        if not hmac.compare_digest(tag, expected):
            raise ValueError("authentication tag is invalid: damaged data or wrong key")
        return aes256_ctr(cipher, key, nonce)

def _dump(path: str | Path, fmt: str, values: dict[str, Any]) -> None:
    Path(path).write_text(
        json.dumps({"format": fmt, **values}, sort_keys=True), encoding="utf-8"
    )

def _load(path: str | Path, fmt: str) -> dict[str, Any]:
    try:
        v = json.loads(Path(path).read_text(encoding="utf-8"))
    except (OSError, UnicodeDecodeError, json.JSONDecodeError) as e:
        raise ValueError("invalid cryptosystem container") from e
    if v.get("format") != fmt:
        raise ValueError(f"expected {fmt} container")
    return v

def save_public_key(path: str | Path, key: PublicKey) -> None:
    _dump(path, "lab04-public-v1", key.to_dict())

def load_public_key(path: str | Path) -> PublicKey:
    try:
        return PublicKey.from_dict(_load(path, "lab04-public-v1"))
    except (KeyError, TypeError, ValueError) as e:
        raise ValueError("unable to load public key") from e

def save_private_key(path: str | Path, key: PrivateKey, password: str) -> None:
    if not password:
        raise ValueError("master password cannot be empty")
    salt = secrets.token_bytes(16)
    aes = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, PBKDF2_ITERATIONS, 32)
    nonce, cipher, tag = _auth_encrypt(
        json.dumps(key.to_dict()).encode(), aes, b"lab04-private-v1"
    )
    _dump(
        path,
        "lab04-private-v1",
        {
            "iterations": PBKDF2_ITERATIONS,
            "salt": _b64(salt),
            "nonce": _b64(nonce),
            "ciphertext": _b64(cipher),
            "tag": _b64(tag),
        },
    )

```

```

def load_private_key(path: str | Path, password: str) -> PrivateKey:
    v = _load(path, "lab04-private-v1")
    try:
        salt, nonce, cipher, tag = (
            _unb64(v[x]) for x in ("salt", "nonce", "ciphertext", "tag")
        )
        aes = hashlib.pbkdf2_hmac(
            "sha256", password.encode(), salt, int(v["iterations"]), 32
        )
        return PrivateKey.from_dict(
            json.loads(
                _auth_decrypt(nonce, cipher, tag, aes, b"lab04-private-v1").decode()
            )
        )
    except (
        KeyError,
        TypeError,
        ValueError,
        UnicodeDecodeError,
        json.JSONDecodeError,
    ) as e:
        raise ValueError(
            "unable to decrypt private key: wrong password or damaged file"
        ) from e

    def _fp(key: PublicKey) -> bytes:
    return hashlib.sha256(f"{key.n}:{key.e}".encode()).digest()

def encrypt_blocks(source: str | Path, destination: str | Path, key: PublicKey) -> None:
    """Блочный RSA: k-1 байт -> k байт; заголовок содержит длину и ключ."""
    plain = Path(source).read_bytes()
    k = key.size_bytes
    b = k - 1
    out = bytearray(
        RAW_HEADER.pack(
            RAW_MAGIC, b, len(plain), _fp(key), hashlib.sha256(plain).digest()
        )
    )
    for i in range(0, len(plain), b):
        out.extend(
            _i2osp(encrypt_integer(_os2ip(plain[i : i + b].ljust(b, b"\0")), key), k)
        )
    Path(destination).write_bytes(out)

    def decrypt_blocks(
        source: str | Path,
        destination: str | Path,

```

```

        key: PrivateKey,
        *,
        use_crt: bool = True,
    ) -> None:
        data = Path(source).read_bytes()
        if len(data) < RAW_HEADER.size:
            raise ValueError("ciphertext is too short")
        magic, b, length, fp, digest = RAW_HEADER.unpack(data[: RAW_HEADER.size])
        k = key.size_bytes
        if magic != RAW_MAGIC or b != k - 1:
            raise ValueError("ciphertext format or RSA key length is incorrect")
        if not hmac.compare_digest(fp, _fp(key.public_key)):
            raise ValueError("ciphertext was encrypted for another key")
        count = math.ceil(length / b)
        if len(data) != RAW_HEADER.size + count * k:
            raise ValueError("ciphertext length is damaged")
        out = bytearray()
        q_inverse = mod_inverse(key.q, key.p) if use_crt and key.p and key.q else None
        for i in range(RAW_HEADER.size, len(data), k):
            v = _os2ip(data[i : i + k])
            if v >= key.n:
                raise ValueError("ciphertext block outside RSA modulus")
            try:
                decrypted = (
                    _decrypt_crt(v, key, q_inverse)
                    if q_inverse is not None
                    else decrypt_integer(v, key, use_crt=False)
                )
                out.extend(_i2osp(decrypted, b))
            except ValueError as e:
                raise ValueError("ciphertext is damaged (invalid plaintext block)") from e
        plaintext = bytes(out[:length])
        if not hmac.compare_digest(hashlib.sha256(plaintext).digest(), digest):
            raise ValueError("ciphertext is damaged (SHA-256 integrity check failed)")
        Path(destination).write_bytes(plaintext)

    def benchmark_file(
        source: str | Path, bits_values: tuple[int, ...] = (1024, 2048, 3072)
    ) -> list[dict[str, float | int]]:
        """Measure key generation, encryption, normal and CRT decryption times.

        Temporary files are used, so the benchmark never overwrites user data.
        Passing a file of roughly 100 KiB produces the table requested in task 3.
        """
        original = Path(source).read_bytes()
        results = []
        with tempfile.TemporaryDirectory(prefix="lab04-rsa-") as directory:
            root = Path(directory)
            plain = root / "input.bin"
            plain.write_bytes(original)
            for bits in bits_values:

```

```

        started = time.perf_counter()
        public, private = generate_key_pair(bits)
        key_time = time.perf_counter() - started
        encrypted = root / f"{bits}.rsa"
        normal = root / f"{bits}.normal"
        crt = root / f"{bits}.crt"
        started = time.perf_counter()
        encrypt_blocks(plain, encrypted, public)
        encrypt_time = time.perf_counter() - started
        started = time.perf_counter()
        decrypt_blocks(encrypted, normal, private, use_crt=False)
        normal_time = time.perf_counter() - started
        started = time.perf_counter()
        decrypt_blocks(encrypted, crt, private, use_crt=True)
        crt_time = time.perf_counter() - started
        if normal.read_bytes() != original or crt.read_bytes() != original:
            raise RuntimeError("RSA benchmark round-trip failed")
        results.append(
            {
                "bits": bits,
                "key_generation_seconds": key_time,
                "encryption_seconds": encrypt_time,
                "decryption_normal_seconds": normal_time,
                "decryption_crt_seconds": crt_time,
            }
        )
    return results

```

```

def encrypt_text(text: str, key: PublicKey) -> str:
    raw = text.encode()
    b = key.size_bytes - 1
    blocks = [
        _b64(
            _i2osp(
                encrypt_integer(_os2ip(raw[i : i + b].ljust(b, b"\0")), key),
                key.size_bytes,
            )
        )
    ]
    for i in range(0, len(raw), b)
    return json.dumps(
        {
            "format": "lab04-text-v1",
            "length": len(raw),
            "key": _b64(_fp(key)),
            "blocks": blocks,
        }
    )

```

```

def decrypt_text(container: str, key: PrivateKey, *, use_crt: bool = True) -> str:

```

```

        try:
            v = json.loads(container)
            b = key.size_bytes - 1
            length = int(v["length"])
            if (
                v["format"] != "lab04-text-v1"
                or not hmac.compare_digest(_unb64(v["key"]), _fp(key.public_key))
                or len(v["blocks"]) != math.ceil(length / b)
            ):
                raise ValueError("wrong key or malformed text")
            raw = b"".join(
                _i2osp(decrypt_integer(_os2ip(_unb64(x)), key, use_crt=use_crt), b)
                for x in v["blocks"]
            )
            return raw[:length].decode()
        except (
            KeyError,
            TypeError,
            ValueError,
            UnicodeDecodeError,
            json.JSONDecodeError,
        ) as e:
            raise ValueError("unable to decrypt text") from e

    def sign(data: bytes, key: PrivateKey) -> bytes:
        return _i2osp(
            decrypt_integer(_os2ip(hashlib.sha256(data).digest()), key), key.size_bytes
        )

    def verify(data: bytes, signature: bytes, key: PublicKey) -> bool:
        return (
            len(signature) == key.size_bytes
            and _os2ip(signature) < key.n
            and encrypt_integer(_os2ip(signature), key)
            == _os2ip(hashlib.sha256(data).digest())
        )

    def sign_file(source: str | Path, destination: str | Path, key: PrivateKey) -> None:
        _dump(
            destination,
            "lab04-signature-v1",
            {"signature": _b64(sign(Path(source).read_bytes(), key))},
        )

    def verify_file(source: str | Path, signature_path: str | Path, key: PublicKey) -> bool:
        try:
            return verify(
                Path(source).read_bytes(),

```

```

        _unb64(_load(signature_path, "lab04-signature-v1")["signature"]),
                key,
            )
        except (OSError, KeyError, TypeError, ValueError):
            return False

def encrypt_file(source: str | Path, destination: str | Path, key: PublicKey) -> None:
    """Гибрид: AES-256-CTR, ключ AES шифруется собственной RSA-OAEP."""
    aes = secrets.token_bytes(32)
    nonce, cipher, tag = _auth_encrypt(Path(source).read_bytes(), aes, b"lab04-file-v1")
    _dump(
        destination,
        "lab04-envelope-v1",
        {
            "encrypted_key": _b64(rsa_oaep_encrypt(aes, key)),
            "nonce": _b64(nonce),
            "ciphertext": _b64(cipher),
            "tag": _b64(tag),
        },
    )

def decrypt_file(source: str | Path, destination: str | Path, key: PrivateKey) -> None:
    v = _load(source, "lab04-envelope-v1")
    try:
        plain = _auth_decrypt(
            _unb64(v["nonce"]),
            _unb64(v["ciphertext"]),
            _unb64(v["tag"]),
            rsa_oaep_decrypt(_unb64(v["encrypted_key"]), key),
            b"lab04-file-v1",
        )
    except (KeyError, TypeError, ValueError) as e:
        raise ValueError(
            "unable to decrypt file: incorrect key or damaged envelope"
        ) from e
    Path(destination).write_bytes(plain)

def demonstrate_textbook_weaknesses(key: PublicKey) -> dict[str, int | bool]:
    c1, c2 = encrypt_integer(2, key), encrypt_integer(3, key)
    return {
        "same_plaintext_same_ciphertext": c1 == encrypt_integer(2, key),
        "multiplicative_property": (c1 * c2) % key.n == encrypt_integer(6, key),
        "m1": 2,
        "m2": 3,
        "c1": c1,
        "c2": c2,
    }

```

Листинг 5.1 — Модуль криптографических операций

```

"""CLI для лабораторной работы №4.

python3 src/main.py genkeys --bits 2048 --pub public.key --priv private.key
python3 src/main.py encrypt --key public.key --in input.bin --out input.rsa
python3 src/main.py decrypt --key private.key --in input.rsa --out recovered.bin
"""

from __future__ import annotations
import argparse, getpass, json, sys, time
    from pathlib import Path
    from rsa_crypto import *

def _password(confirm=False):
    value = getpass.getpass("Мастер-пароль: ")
    if confirm and value != getpass.getpass("Повторите мастер-пароль: "):
        raise ValueError("пароли не совпадают")
    return value

def _file(p, private=False):
    p.add_argument("--key", required=True)
    p.add_argument("--in", dest="source", required=True)
    p.add_argument("--out", dest="destination", required=True)

def build_parser():
    p = argparse.ArgumentParser(
        description="ЛР №4: RSA, КТО, подпись и гибридная схема"
    )
    s = p.add_subparsers(dest="command", required=True)
    x = s.add_parser("genkeys", aliases=["keygen"])
    x.add_argument(
        "--bits", type=int, default=3072, help="длина модуля; для варианта 15 – 3072"
    )
    x.add_argument("--pub", required=True)
    x.add_argument("--priv", required=True)
    for name, priv in (
        ("encrypt", False),
        ("decrypt", True),
        ("hybrid-encrypt", False),
        ("hybrid-decrypt", True),
        ("sign", True),
        ("verify", False),
    ):
        _file(s.add_parser(name), priv)
    x = s.add_parser("encrypt-int")
    x.add_argument("--key", required=True)
    x.add_argument("message", type=int)
    x = s.add_parser("decrypt-int")
    x.add_argument("--key", required=True)
    x.add_argument("ciphertext", type=int)

```

```

        x = s.add_parser("encrypt-text")
        x.add_argument("--key", required=True)
        x.add_argument("text")
        x.add_argument("--out", required=True)
        x = s.add_parser("decrypt-text")
        x.add_argument("--key", required=True)
        x.add_argument("--in", dest="source", required=True)
        x.add_argument("--out")
        x = s.add_parser("crt-time")
        x.add_argument("--key", required=True)
        x.add_argument("ciphertext", type=int)
        x.add_argument("--repetitions", type=int, default=3)
        x = s.add_parser(
            "benchmark", help="замеры для файла (по умолчанию: 1024, 2048, 3072 бит)"
        )
        x.add_argument("--in", dest="source", required=True)
        x.add_argument("--bits", type=int, nargs="+", default=[1024, 2048, 3072])
        x.add_argument("--out")
        x = s.add_parser("weaknesses")
        x.add_argument("--key", required=True)
        s.add_parser("variant15", help="проверить расчёт p=83, q=19, e=13, M=222")
        return p

def main():
    a = build_parser().parse_args()
    try:
        if a.command in ("genkeys", "keygen"):
            t = time.perf_counter()
            pub, priv = generate_key_pair(a.bits)
            save_public_key(a.pub, pub)
            save_private_key(a.priv, priv, _password(True))
            print(f"Ключи созданы за {time.perf_counter()-t:.3f} с.")
            elif a.command == "encrypt":
                encrypt_blocks(a.source, a.destination, load_public_key(a.key))
                print("Файл зашифрован учебным RSA.")
            elif a.command == "decrypt":
                decrypt_blocks(
                    a.source, a.destination, load_private_key(a.key, _password())
                )
                print("Файл расшифрован.")
            elif a.command == "hybrid-encrypt":
                encrypt_file(a.source, a.destination, load_public_key(a.key))
                print("Гибридный контейнер создан.")
            elif a.command == "hybrid-decrypt":
                decrypt_file(a.source, a.destination, load_private_key(a.key, _password()))
                print("Гибридный контейнер расшифрован.")
            elif a.command == "sign":
                sign_file(a.source, a.destination, load_private_key(a.key, _password()))
                print("Подпись создана.")
            elif a.command == "verify":
                ok = verify_file(a.source, a.destination, load_public_key(a.key))

```

```

print("Подпись корректна." if ok else "Подпись некорректна.")
    return 0 if ok else 1
    elif a.command == "encrypt-int":
        print(encrypt_integer(a.message, load_public_key(a.key)))
    elif a.command == "decrypt-int":
        print(decrypt_integer(a.ciphertext, load_private_key(a.key, _password())))
    elif a.command == "encrypt-text":
        Path(a.out).write_text(
            encrypt_text(a.text, load_public_key(a.key)), encoding="utf-8"
        )
        print("Текст зашифрован.")
    elif a.command == "decrypt-text":
        text = decrypt_text(
            Path(a.source).read_text(encoding="utf-8"),
            load_private_key(a.key, _password()),
        )
        if a.out:
            Path(a.out).write_text(text, encoding="utf-8")
        else:
            print(text)
    elif a.command == "crt-time":
        normal, crt = rsa_decrypt_timing(
            a.ciphertext, load_private_key(a.key, _password()), a.repetitions
        )
        print(
            json.dumps(
                {
                    "ordinary_seconds": normal,
                    "crt_seconds": crt,
                    "speedup": normal / crt if crt else None,
                }
            )
        )
    elif a.command == "benchmark":
        result = benchmark_file(a.source, tuple(a.bits))
        rendered = json.dumps(result, ensure_ascii=False, indent=2)
        if a.out:
            Path(a.out).write_text(rendered + "\n", encoding="utf-8")
            print(rendered)
    elif a.command == "variant15":
        print(json.dumps(solve_variant_15(), ensure_ascii=False, indent=2))
    else:
        print(
            json.dumps(
                demonstrate_textbook_weaknesses(load_public_key(a.key)), indent=2
            )
        )
    return 0
except (OSError, ValueError) as e:
    print(f"Ошибка: {e}", file=sys.stderr)
    return 2

```

```
if __name__ == "__main__":  
    raise SystemExit(main())
```

Листинг 5.2 — Модуль консольного интерфейса